

Lectura y Escritura de Archivos en Java

Guía completa con ejemplos, parámetros y valores de retorno

[java.io](#) · [java.nio.file](#) · [Streams](#) · [Buenas prácticas](#)

Contenido

- 1. Introducción y jerarquía de clases
- 2. Clases fundamentales de `java.io`
- 3. API moderna: `java.nio.file`
- 4. Lectura de archivos de texto
- 5. Escritura de archivos de texto
- 6. Archivos binarios
- 7. Manejo de excepciones
- 8. Resumen comparativo
- 9. Buenas prácticas

1. Introducción v Jerarquía de Clases

Java ofrece dos APIs para trabajar con archivos. La primera, **java.io**, existe desde Java 1.0 y trabaja con streams (flujos de bytes o caracteres). La segunda, **java.nio.file**, introducida en Java 7, es más moderna, expresiva y recomendada para proyectos nuevos. Ambas conviven y se complementan.

Jerarquía simplificada de clases

Elemento	Descripción
InputStream	Clase abstracta base para leer bytes
OutputStream	Clase abstracta base para escribir bytes
Reader	Clase abstracta base para leer caracteres (texto)
Writer	Clase abstracta base para escribir caracteres (texto)
FileInputStream	Lee bytes directamente de un archivo
FileOutputStream	Escribe bytes directamente en un archivo
FileReader	Lee caracteres de un archivo de texto
FileWriter	Escribe caracteres en un archivo de texto
BufferedReader	Envuelve un Reader y añade buffer + readLine()
BufferedWriter	Envuelve un Writer y añade buffer + newLine()
PrintWriter	Envuelve un Writer y añade print/println/printf
Path	Representa la ruta de un archivo o directorio (nio)
Files	Clase de utilidades estáticas para operaciones de archivo (nio)

■ *try-with-resources (try(...){}) cierra automáticamente cualquier recurso que implemente AutoCloseable. Úsalo SIEMPRE con streams.*

2. Clases Fundamentales de java.io

2.1 FileReader y FileWriter

FileReader lee caracteres de un archivo. **FileWriter** los escribe. Son convenientes para archivos pequeños, pero no tienen buffer propio, por lo que cada lectura/escritura implica una llamada al sistema operativo.

Constructor y parámetros

Elemento	Descripción
FileReader(String)	Recibe la ruta del archivo como String
FileReader(File)	Recibe un objeto File que apunta al archivo
FileReader(Path)	Recibe un Path (disponible desde Java 11)
FileWriter(String)	Ruta del archivo donde se escribirá
FileWriter(String, boolean append)	Si append=true, no borra el contenido existente

Métodos principales

Elemento	Descripción
int read()	Lee un solo carácter. Retorna -1 al llegar al final del archivo
int read(char[] buf)	Lee hasta buf.length caracteres en el array. Retorna cantidad leída o -1
void write(int c)	Escribe un solo carácter
void write(String s)	Escribe toda la cadena s
void write(char[] buf)	Escribe todos los caracteres del array
void close()	Libera el recurso. Lanza IOException si falla

```
import java.io.*;

// Lectura carácter a carácter (no recomendado para archivos grandes)
try (FileReader fr = new FileReader("datos.txt")) {
    int character;
    while ((character = fr.read()) != -1) { // read() devuelve int (-1 = EOF)
        System.out.print((char) character); // cast a char para mostrar
    }
} catch (IOException e) {
    e.printStackTrace();
}

// Escritura con FileWriter (append = true no borra el archivo)
try (FileWriter fw = new FileWriter("salida.txt", true)) {
    fw.write("Hola Mundo"); // void: no devuelve nada
    fw.write(System.lineSeparator()); // salto de línea portable
}
```

■ Retorna: read() → int (carácter como entero, o -1); write() → void

2.2 BufferedReader y BufferedWriter

Añaden un **buffer interno** (por defecto 8 192 caracteres) que reduce drásticamente las llamadas al sistema. `BufferedReader` añade el método **`readLine()`**, fundamental para procesar texto línea a línea.

Constructor

Elemento	Descripción
<code>BufferedReader(Reader r)</code>	Envuelve cualquier <code>Reader</code> con buffer de 8 192 chars
<code>BufferedReader(Reader r, int size)</code>	Igual, pero con tamaño de buffer personalizado
<code>BufferedWriter(Writer w)</code>	Envuelve cualquier <code>Writer</code> con buffer de 8 192 chars
<code>BufferedWriter(Writer w, int size)</code>	Buffer personalizado

Métodos clave

Elemento	Descripción
<code>String readLine()</code>	Lee una línea completa SIN el salto <code>\n</code> . Retorna <code>null</code> al final
<code>Stream<String> lines()</code>	Retorna un <code>Stream</code> lazy de todas las líneas del archivo
<code>void newLine()</code>	Escribe el separador de línea del SO actual (<code>\n</code> o <code>\r\n</code>)
<code>void flush()</code>	Fuerza la escritura del buffer al disco aunque no esté lleno

```
import java.io.*;

// Lectura eficiente línea a línea
try (BufferedReader br = new BufferedReader(new FileReader("log.txt"))) {
    String linea;
    int numero = 1;
    // readLine() retorna String con la línea, o null al llegar al final
    while ((linea = br.readLine()) != null) {
        System.out.printf("%3d: %s\n", numero++, linea);
    }
}

// Con Stream de Java 8+ (más conciso)
try (BufferedReader br = new BufferedReader(new FileReader("log.txt"))) {
    br.lines() // Stream<String> - carga lazy
        .filter(l -> !l.isBlank()) // filtra líneas vacías
        .forEach(System.out::println);
}

// Escritura con BufferedWriter
try (BufferedWriter bw = new BufferedWriter(new FileWriter("out.txt"))) {
    bw.write("Primera línea");
    bw.newLine(); // void - escribe \n o \r\n según el SO
    bw.write("Segunda línea");
    bw.flush(); // void - vacía el buffer al disco
}
```

■ Retorna: `readLine()` → `String` | `null` · `lines()` → `Stream` · `write/newLine/flush` → `void`

2.3 PrintWriter

Hereda de `Writer` y añade métodos de impresión formateada. Es el `Writer` más cómodo para generar texto, similar a `System.out`.

Elemento	Descripción
<code>void print(T valor)</code>	Escribe cualquier tipo sin salto de línea
<code>void println(T valor)</code>	Escribe el valor y añade salto de línea
<code>void printf(String fmt, ...)</code>	Escritura con formato estilo C. Retorna <code>PrintWriter</code>
<code>boolean checkError()</code>	Retorna <code>true</code> si ocurrió algún error de escritura

```
import java.io.*;

try (PrintWriter pw = new PrintWriter(new BufferedWriter(new FileWriter("reporte.txt"))))
{
    pw.println("=== Reporte de Ventas ==="); // void
    pw.printf("Total: %.2f EUR%n", 1234.5); // void, formato tipo C
    pw.print(true); // escribe 'true' sin salto de línea
    pw.println(); // solo salto de línea

    if (pw.checkError()) { // boolean: true si hubo error
        System.err.println("Error al escribir el archivo");
    }
}
```

■ Retorna: `print/println/printf` → `void` · `checkError()` → `boolean`

3. API Moderna: java.nio.file

El paquete **java.nio.file** (New I/O, Java 7+) introduce las clases **Path**, **Paths**, **Files** y **StandardOpenOption**. Simplifican enormemente las operaciones comunes y son la opción recomendada en proyectos modernos.

3.1 La clase Path

Path representa la ruta de un archivo o directorio. Es inmutable. Se obtiene mediante el método de fábrica **Path.of()** (Java 11+) o **Paths.get()** (Java 7+).

Elemento	Descripción
Path.of(String)	Crea un Path a partir de una cadena de ruta. Retorna Path
Path.of(String, String...)	Une segmentos de ruta. Retorna Path
path.resolve(String otro)	Combina este Path con otro relativo. Retorna Path
path.getParent()	Retorna el Path del directorio padre, o null si no existe
path.getFileName()	Retorna el último elemento de la ruta como Path
path.toAbsolutePath()	Retorna la ruta absoluta. Retorna Path
path.toString()	Convierte el Path a String con el separador del SO
path.toFile()	Convierte el Path al antiguo java.io.File

```
import java.nio.file.Path;

Path p1 = Path.of("documentos/notas.txt"); // relativo
Path p2 = Path.of("/home/usuario", "archivo.txt"); // absoluto con segmentos

System.out.println(p1.getFileName()); // Path → 'notas.txt'
System.out.println(p1.getParent()); // Path → 'documentos'
System.out.println(p1.toAbsolutePath()); // Path → ruta absoluta completa

Path base = Path.of("/proyectos");
Path completo = base.resolve("src/Main.java"); // Path → /proyectos/src/Main.java
```

■ Retorna: **Path.of()** → **Path** · **resolve()** → **Path** · **toString()** → **String**

3.2 La clase Files – Métodos de utilidad

Files es una clase de métodos estáticos. Cubre casi todas las operaciones de archivo que necesitarás en el día a día.

Elemento	Descripción
Files.exists(Path)	Comprueba si el archivo/dir existe. Retorna boolean
Files.isDirectory(Path)	Retorna true si la ruta es un directorio
Files.isReadable(Path)	Retorna true si el proceso puede leer el archivo
Files.size(Path)	Retorna el tamaño en bytes como long
Files.createFile(Path)	Crea un archivo vacío. Lanza exception si ya existe. Retorna Path

Files.createDirectories(Path)	Crea el directorio y todos los padres necesarios. Retorna Path
Files.delete(Path)	Elimina el archivo/dir. Lanza NoSuchFileException si no existe
Files.deleteIfExists(Path)	Elimina si existe. Retorna boolean (true si borró)
Files.copy(Path src, Path dst)	Copia un archivo. Retorna Path destino
Files.move(Path src, Path dst)	Mueve/renombra un archivo. Retorna Path destino
Files.list(Path dir)	Retorna Stream<Path> con el contenido inmediato del dir
Files.walk(Path dir)	Retorna Stream<Path> recorriendo todo el árbol de directorios

```
import java.nio.file.*;

Path archivo = Path.of("config.txt");

// Comprobaciones - todas retornan boolean
boolean existe = Files.exists(archivo);
boolean esDir = Files.isDirectory(archivo);
boolean legible = Files.isReadable(archivo);

// Tamaño - retorna long (bytes)
long bytes = Files.size(archivo);
System.out.println(bytes + " bytes");

// Crear directorios anidados (no falla si ya existen)
Path directorios = Path.of("proyectos/java/src");
Files.createDirectories(directorios); // retorna Path

// Copiar con opción de sobreescritura
Files.copy(archivo,
Path.of("config_backup.txt"),
StandardCopyOption.REPLACE_EXISTING); // retorna Path destino

// Listar archivos de un directorio
try (Stream<Path> stream = Files.list(Path.of("."))) {
stream.filter(Files::isRegularFile) // solo archivos, no dirs
.forEach(System.out::println);
}
```

■ Retorna: exists/isDirectory/isReadable → boolean · size() → long · copy/move/createFile → Path

4. Lectura de Archivos de Texto

4.1 Files.readString() — Java 11+

Carga todo el contenido del archivo en un único String. Ideal para archivos pequeños (configuraciones, plantillas). Para archivos grandes puede consumir demasiada memoria.

Elemento	Descripción
Files.readString(Path)	Lee todo el archivo en UTF-8. Retorna String
Files.readString(Path, Charset)	Lee con el charset especificado. Retorna String

```
import java.nio.file.*;
import java.nio.charset.StandardCharsets;
import java.io.IOException;

Path ruta = Path.of("config.json");

// Versión simple - usa UTF-8 por defecto
// Retorna: String con todo el contenido
String contenido = Files.readString(ruta);

// Con charset explícito (buena práctica)
String contenidoISO = Files.readString(ruta, StandardCharsets.ISO_8859_1);

System.out.println(contenido);
System.out.println("Longitud: " + contenido.length() + " caracteres");
```

■ Retorna: Files.readString() → String (todo el contenido del archivo)

4.2 Files.readAllLines() — Lista de líneas

Lee todas las líneas y las devuelve como **List<String>**. Conveniente cuando necesitas acceso por índice o contar líneas.

```
import java.nio.file.*;
import java.util.List;

// Retorna: List<String> - una entrada por línea (sin \n)
List<String> lineas = Files.readAllLines(Path.of("datos.csv"));

System.out.println("Total de líneas: " + lineas.size()); // int

// Acceso por índice
String cabecera = lineas.get(0); // primera línea

// Procesar desde la línea 1 (saltar cabecera)
lineas.stream()
    .skip(1)
    .map(linea -> linea.split(",")) // dividir por coma
    .forEach(partes -> System.out.println(partes[0]));
```

■ Retorna: Files.readAllLines() → List

4.3 Files.lines() — Stream lazy (archivos grandes)

A diferencia de `readAllLines()`, **`Files.lines()`** devuelve un `Stream` que lee el archivo a medida que se consume. Es ideal para archivos de millones de líneas porque no carga todo en memoria.

```
import java.nio.file.*;
import java.util.stream.Stream;

// Retorna: Stream<String> - lazy, lee bajo demanda
// IMPORTANTE: cerrar el Stream cierra también el archivo
try (Stream<String> stream = Files.lines(Path.of("gigante.log"))) {
    long errores = stream
        .filter(l -> l.contains("ERROR")) // solo líneas con ERROR
        .peek(System.out::println) // imprimir cada una
        .count(); // long: cantidad de errores

    System.out.println("Errores encontrados: " + errores);
}
```

■ Retorna: `Files.lines()` → `Stream` · `stream.count()` → `long`

4.4 `BufferedReader.readLine()` — Control total

Cuando necesitas procesar línea a línea con lógica compleja, acceder a números de línea o integrar con código heredado de `java.io`.

```
import java.io.*;
import java.nio.file.*;
import java.nio.charset.StandardCharsets;

// Files.newBufferedReader crea un BufferedReader con charset
try (BufferedReader br = Files.newBufferedReader(
    Path.of("archivo.txt"), StandardCharsets.UTF_8)) {

    String linea;
    int numLinea = 0;

    // readLine() retorna String con la línea sin \n
    // retorna null al llegar al final del archivo
    while ((linea = br.readLine()) != null) {
        numLinea++;
        if (linea.startsWith("#")) continue; // saltar comentarios
        System.out.printf("[%d] %s\n", numLinea, linea);
    }
    System.out.println("Líneas procesadas: " + numLinea);
}
```

■ Retorna: `readLine()` → `String` (la línea sin salto) o `null` si llegó al final

4.5 `Scanner` — Parseo de tipos

`Scanner` es útil cuando el archivo contiene números, booleans u otros tipos que necesitas parsear directamente sin convertir `Strings` manualmente.

Elemento	Descripción
<code>hasNextLine()</code>	Retorna boolean: true si quedan más líneas
<code>nextLine()</code>	Lee la línea completa. Retorna <code>String</code>

nextInt()	Lee el siguiente token como int
nextDouble()	Lee el siguiente token como double
next()	Lee el siguiente token (separado por espacio). Retorna String
useDelimiter(String)	Cambia el delimitador (por defecto espacio en blanco)

```
import java.util.Scanner;
import java.io.File;
import java.io.FileNotFoundException;

// Archivo de ejemplo con datos: 'Juan 28 9.5'
try (Scanner sc = new Scanner(new File("personas.txt"))) {
    while (sc.hasNextLine()) { // boolean
        String nombre = sc.next(); // String: primer token
        int edad = sc.nextInt(); // int: segundo token
        double nota = sc.nextDouble(); // double: tercer token
        System.out.printf("%s tiene %d años, nota %.1f%n",
            nombre, edad, nota);
    }
} catch (FileNotFoundException e) {
    System.err.println("Archivo no encontrado: " + e.getMessage());
}
```

■ Retorna: `nextLine()` → String · `nextInt()` → int · `nextDouble()` → double · `hasNextLine()` → boolean

5. Escritura de Archivos de Texto

5.1 Files.writeString() — Java 11+

La forma más directa de escribir texto. Si el archivo existe lo sobrescribe; si no existe, lo crea. Usa opciones (**StandardOpenOption**) para cambiar el comportamiento.

Elemento	Descripción
StandardOpenOption.CREATE	Crea el archivo si no existe (comportamiento por defecto)
StandardOpenOption.APPEND	Añade al final sin borrar el contenido existente
StandardOpenOption.TRUNCATE_EXISTING	Borra el contenido al abrir (defecto en escritura)
StandardOpenOption.CREATE_NEW	Crea el archivo; lanza excepción si ya existe

```
import java.nio.file.*;
import java.nio.charset.StandardCharsets;

Path archivo = Path.of("salida.txt");

// Escribir (crea o sobrescribe). Retorna: Path del archivo escrito
Path resultado = Files.writeString(archivo, "Hola, mundo!\n");
System.out.println("Escrito en: " + resultado); // salida.txt

// Añadir al final (APPEND) con charset explícito. Retorna: Path
Files.writeString(archivo,
"Segunda línea\n",
StandardCharsets.UTF_8,
StandardOpenOption.APPEND);

// Crear solo si NO existe (lanza FileAlreadyExistsException si existe)
Files.writeString(Path.of("nuevo.txt"),
"Solo si es nuevo",
StandardOpenOption.CREATE_NEW);
```

■ Retorna: Files.writeString() → Path (la ruta del archivo escrito)

5.2 Files.write() — Lista de líneas

Escribe una colección de líneas. Java añade automáticamente el separador de línea del sistema operativo entre cada elemento.

```
import java.nio.file.*;
import java.util.List;

List<String> lineas = List.of(
"nombre,edad,ciudad",
"Ana,25,Madrid",
"Luis,30,Barcelona",
"Sara,22,Valencia"
);

// Retorna: Path del archivo escrito
Path csv = Files.write(Path.of("personas.csv"), lineas);
```

```
// Con charset explícito y opciones
Files.write(Path.of("log.txt"),
List.of("Inicio: " + java.time.LocalDateTime.now()),
StandardCharsets.UTF_8,
StandardOpenOption.CREATE,
StandardOpenOption.APPEND);
```

■ **Retorna:** `Files.write()` → `Path` (la ruta del archivo)

5.3 `BufferedWriter` — Escritura eficiente con buffer

```
import java.io.*;
import java.nio.file.*;
import java.nio.charset.StandardCharsets;

// Files.newBufferedWriter crea el archivo y lo envuelve en BufferedWriter
try (BufferedWriter bw = Files.newBufferedWriter(
Path.of("reporte.txt"), StandardCharsets.UTF_8)) {

    bw.write("=== Reporte ==="); // void: escribe String
    bw.newLine(); // void: \n o \r\n según SO
    bw.write("Total: 42");
    bw.newLine();

    // Forzar escritura del buffer al disco (opcional con try-resources)
    bw.flush(); // void
} // close() automático vacía el buffer y cierra
```

■ **Retorna:** `write()` → `void` · `newLine()` → `void` · `flush()` → `void`

5.4 `PrintWriter` — Escritura formateada

```
import java.io.*;

// PrintWriter acepta FileWriter, BufferedWriter, OutputStream, etc.
try (PrintWriter pw = new PrintWriter(
new BufferedWriter(new FileWriter("factura.txt")))) {

    pw.println("=== FACTURA ==="); // void + salto de línea
    pw.print("Cliente: "); // void sin salto de línea
    pw.println("Empresa S.A.");
    pw.printf("Total: %, .2f EUR%n", 3456.789); // void, formato
    pw.printf("Fecha: %tF%n", new java.util.Date()); // fecha ISO

    // checkError() retorna boolean: true si hubo algún error
    if (pw.checkError()) {
        throw new RuntimeException("Error al escribir la factura");
    }
}
```

■ **Retorna:** `println/print/printf` → `void` · `checkError()` → `boolean`

6. Archivos Binarios

Para imágenes, audio, videos, archivos comprimidos o cualquier dato no textual, se trabaja con bytes crudos usando `InputStream/OutputStream` o los métodos binarios de `Files`.

6.1 `Files.readAllBytes()` y `Files.write()`

Elemento	Descripción
<code>Files.readAllBytes(Path)</code>	Lee todo el archivo como <code>byte[]</code> . Retorna <code>byte[]</code>
<code>Files.write(Path, byte[])</code>	Escribe un <code>byte[]</code> en el archivo. Retorna <code>Path</code>

```
import java.nio.file.*;

// Leer archivo binario completo en memoria
// Retorna: byte[] con los bytes del archivo
byte[] imagen = Files.readAllBytes(Path.of("foto.jpg"));
System.out.println("Tamaño: " + imagen.length + " bytes");

// Copiar archivo (leer + escribir bytes)
byte[] datos = Files.readAllBytes(Path.of("original.bin"));
Files.write(Path.of("copia.bin"), datos); // Retorna Path

// ■ NO usar con archivos muy grandes: carga TODO en memoria
```

■ Retorna: `readAllBytes()` → `byte[]` · `write(Path, byte[])` → `Path`

6.2 `InputStream` y `OutputStream` con buffer

Para archivos grandes (videos, backups, logs masivos) se usa un buffer para procesar en trozos sin agotar la memoria.

Elemento	Descripción
<code>int read(byte[] buf)</code>	Lee hasta <code>buf.length</code> bytes. Retorna bytes leídos, o -1 al final
<code>void write(byte[] buf, int off, int len)</code>	Escribe <code>len</code> bytes desde la posición <code>off</code> del array
<code>long transferTo(OutputStream out)</code>	Copia todos los bytes al <code>OutputStream</code> destino. Retorna bytes copiados

```
import java.io.*;
import java.nio.file.*;

Path origen = Path.of("video_original.mp4");
Path destino = Path.of("video_copia.mp4");

// Copia eficiente con buffer de 8 KB
try (InputStream in = Files.newInputStream(origen);
    OutputStream out = Files.newOutputStream(destino)) {

    byte[] buffer = new byte[8192]; // 8 KB de buffer
    int bytesLeidos;

    // read() retorna int: bytes leídos en este ciclo, o -1 al terminar
    while ((bytesLeidos = in.read(buffer)) != -1) {
        // Escribe exactamente los bytes que se leyeron
        out.write(buffer, 0, bytesLeidos);
    }
}
```

```

out.write(buffer, 0, bytesLeídos); // void
}
}

// Alternativa con transferTo (Java 9+) - más conciso
// transferTo retorna long: total de bytes copiados
try (InputStream in = Files.newInputStream(origen);
    OutputStream out = Files.newOutputStream(destino)) {
    long copiados = in.transferTo(out);
    System.out.println(copiados + " bytes copiados");
}

```

■ Retorna: `read(byte[])` → `int` (bytes leídos o -1) · `write()` → `void` · `transferTo()` → `long`

6.3 DataInputStream / DataOutputStream — Tipos primitivos

Permiten leer y escribir tipos Java (`int`, `double`, `boolean`, `String`) directamente en formato binario portable.

```

import java.io.*;

// Escribir datos estructurados en binario
try (DataOutputStream dos = new DataOutputStream(
    new BufferedOutputStream(new FileOutputStream("datos.bin")))) {

    dos.writeUTF("Juan García"); // void - String en formato modificado UTF-8
    dos.writeInt(28); // void - entero en 4 bytes
    dos.writeDouble(9.75); // void - double en 8 bytes
    dos.writeBoolean(true); // void - boolean en 1 byte
}

// Leer los datos en el mismo orden
try (DataInputStream dis = new DataInputStream(
    new BufferedInputStream(new FileInputStream("datos.bin")))) {

    String nombre = dis.readUTF(); // String
    int edad = dis.readInt(); // int
    double nota = dis.readDouble(); // double
    boolean activo = dis.readBoolean(); // boolean

    System.out.printf("%s, %d, %.2f, %b\n", nombre, edad, nota, activo);
}

```

■ Retorna: `readUTF()` → `String` · `readInt()` → `int` · `readDouble()` → `double` · `readBoolean()` → `boolean`

7. Manejo de Excepciones

Todas las operaciones de I/O pueden lanzar excepciones. Java las divide en checked exceptions (debes declarar o capturar) y runtime exceptions.

Elemento	Descripción
IOException	Excepción base de todos los errores de I/O
NoSuchFileException	El archivo o directorio no existe (subclase de IOException)
FileAlreadyExistsException	El archivo ya existe (en operaciones CREATE_NEW)
AccessDeniedException	Sin permisos para leer/escribir/borrar
DirectoryNotEmptyException	Intentar borrar un directorio que tiene contenido
FileNotFoundException	Versión de java.io (heredado)
EOFException	Se alcanzó el fin del stream antes de lo esperado (binario)

```
import java.nio.file.*;
import java.io.IOException;

Path ruta = Path.of("archivo.txt");

try {
    String contenido = Files.readString(ruta);
    System.out.println(contenido);
} catch (NoSuchFileException e) {
    // El archivo no existe: e.getFile() da la ruta como String
    System.err.println("No existe: " + e.getFile());
} catch (AccessDeniedException e) {
    // Sin permisos
    System.err.println("Acceso denegado: " + e.getFile());
} catch (IOException e) {
    // Cualquier otro error de I/O
    System.err.println("Error de I/O: " + e.getMessage());
    e.printStackTrace(); // traza completa para depuración
}

// Patrón: crear el archivo si no existe antes de leer
if (!Files.exists(ruta)) {
    Files.createFile(ruta); // crea vacío
    System.out.println("Archivo creado");
}
```

■ *Ordena los catch de más específico a más general. NoSuchFileException antes de IOException, nunca al revés.*

8. Resumen Comparativo

Situación	Clase/Método recomendado	Retorna
Archivo de texto pequeño (leer)	Files.readString(Path)	String
Texto en lista de líneas (leer)	Files.readAllLines(Path)	List<String>
Archivo grande línea a línea	Files.lines(Path)	Stream<String>
Control total línea a línea	BufferedReader.readLine()	String / null
Parsear tipos (int, double...)	Scanner(File)	int, double, String...
Escribir texto simple	Files.writeString(Path, String)	Path
Escribir lista de líneas	Files.write(Path, Iterable)	Path
Escritura eficiente en bucle	BufferedWriter.write()	void
Escritura con formato	PrintWriter.printf()	void
Archivo binario pequeño	Files.readAllBytes / write(byte[])	byte[] / Path
Archivo binario grande	InputStream + buffer + OutputStream	int / void / long
Tipos primitivos binarios	DataInputStream / DataOutputStream	int, double, String...

9. Buenas Prácticas

✓ Usa siempre try-with-resources

Garantiza el cierre del stream incluso si se lanza una excepción. Sin esto puedes dejar archivos bloqueados o descriptores abiertos.

✓ Especifica el charset explícitamente

Usa `StandardCharsets.UTF_8` en lugar de depender del charset del sistema. Evita sorpresas al mover el código entre Windows, Linux y macOS.

✓ Prefiere `java.nio.file` sobre `java.io`

`Files`, `Path` y `Paths` ofrecen APIs más expresivas, mejores mensajes de error y más funcionalidades que las clases antiguas de `java.io`.

✓ Usa buffer para archivos grandes

`BufferedReader/BufferedWriter` reducen las llamadas al sistema operativo. Para binarios, usa un buffer de al menos 8 KB (`byte[8192]`).

✓ No cargues archivos grandes en memoria

Evita `Files.readAllBytes` o `Files.readAllLines` con archivos de muchos MB. Usa `Files.lines()` o streams con buffer en su lugar.

✓ Valida la existencia antes de operar

Usa `Files.exists()` o captura `NoSuchFileException`. Lanzar y capturar la excepción es más seguro en entornos concurrentes (TOCTOU).

✓ Cierra con `flush()` antes si es necesario

`BufferedWriter` y `PrintWriter` tienen buffer interno. Si no usas try-with-resources, llama `flush()` antes de `close()` para no perder datos.

✓ Usa `Path.of()` en vez de `new File()`

`Path.of()` es más moderno, inmutable, compatible con `Files` y produce código más legible. `File` solo es necesario para APIs antiguas.

Ejemplo integrador — Leer CSV y escribir resultado

```
import java.nio.file.*;
import java.nio.charset.StandardCharsets;
import java.io.*;
import java.util.*;
import java.util.stream.*;

public class ProcesadorCSV {
    public static void main(String[] args) throws IOException {
```

```
Path entrada = Path.of("ventas.csv");
Path salida = Path.of("resumen.txt");

// Leer todas las líneas (salvo cabecera)
// Files.readAllLines → List<String>
List<String> lineas = Files.readAllLines(entrada, StandardCharsets.UTF_8);

// Procesar: formato nombre,importe
DoubleSummaryStatistics stats = lineas.stream()
    .skip(1) // saltar cabecera
    .map(l -> l.split(","))
    .filter(p -> p.length == 2)
    .mapToDouble(p -> Double.parseDouble(p[1].trim()))
    .summaryStatistics(); // DoubleSummaryStatistics

// Construir resumen
List<String> resultado = List.of(
    "=== RESUMEN DE VENTAS ===",
    "Registros : " + stats.getCount(), // long
    String.format("Total : %.2f EUR", stats.getSum()),
    String.format("Promedio : %.2f EUR", stats.getAverage()),
    String.format("Máximo : %.2f EUR", stats.getMax()),
    String.format("Mínimo : %.2f EUR", stats.getMin())
);

// Escribir resultado
// Files.write → Path (la ruta del archivo creado)
Path escritoEn = Files.write(salida, resultado, StandardCharsets.UTF_8);
System.out.println("Resumen guardado en: " + escritoEn);
}
}
```